TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG-HCM

KHOA CÔNG NGHỆ THÔNG TIN

CSC15006 – NHẬP MÔN XỬ LÝ NGÔN NGỮ TỰ NHIÊN

ĐỒ ÁN CUỐI KỲ: FINE-TUNING MODEL TEXT TO SPEECH

Lớp 23CNTThuc1

Sinh viên:

23127223 – Nguyễn Lê Thiên Lý
23127230 – Đặng Lữ Kiều My
23127238 – Trần Hoài Thiện Nhân
23127239 – Lê Nguyên Nhật

Giáo viên hướng dẫn:

TS. Nguyễn Hồng Bửu Long
PGS.TS. Đinh Điền
Anh Tiến Vinh
Thầy Lương An Vinh

Hồ Chí Minh, tháng 12-2025

Mục lục

1 Tìm hiểu về mô hình Coqui XTTS-v2 . . . . . **5**
1.1 Tổng quan và Lý do lựa chọn mô hình 5
1.2 Kiến trúc hệ thống của XTTS-v2 5
1.3 Cơ chế Sao chép giọng nói xuyên ngôn ngữ (Cross-lingual Voice Cloning) 6
1.4 Quy trình huấn luyện (Training Pipeline) 6

2 Fine-tuning trên model anhnh2002/vnTTS **7**
2.1 Chuẩn bị dữ liệu và Tiền xử lý 7
2.2 Quy trình Tinh chỉnh (Training Pipeline) 7
2.2.1 Tinh chỉnh DVAE (Giai đoạn tùy chọn) 7
2.2.2 Tinh chỉnh GPT (Giai đoạn quan trọng nhất) 8
2.3 Tối ưu hóa tham số huấn luyện 9
2.3.1 Điều chỉnh cho DVAE Finetuning 9
2.3.2 Điều chỉnh cho GPT Finetuning 9

3 Dữ liệu huấn luyện (Dataset) **11**
3.1 Các bộ dữ liệu sử dụng 11
3.2 Quy trình xử lý dữ liệu 11
3.3 Lưu trữ dữ liệu trên Kaggle 11

4 Đánh giá mô hình **12**
4.1 Tiêu chí và Phương pháp đánh giá 12
4.2 Kết quả thực nghiệm 12
4.3 Đánh giá cảm quan (Subjective Evaluation) 12
4.3.1 Phương pháp thực hiện 13
4.3.2 Kết quả đánh giá MOS 13
4.4 Phân tích kết quả 14
4.4.1 Về độ giống giọng (Speaker Similarity) 14
4.4.2 Về độ rõ nghĩa (Intelligibility) 14
4.5 Đánh giá qua biểu đồ phổ (Spectrogram) 14
4.6 Phân tích chi tiết lỗi sai (Error Analysis) 15
4.7 Kết luận chương 4 16
4.8 Thiết lập môi trường và Quy trình thực nghiệm 16
4.8.1 Cài đặt môi trường 16
4.8.2 Dữ liệu đầu vào 17
4.8.3 Quy trình đánh giá tự động 17

5 Ứng dụng di động trên Android **19**
5.1 Kiến trúc Hệ thống Client-Server 19
5.1.1 Cấu trúc Ứng dụng Flutter 19
5.2 Cơ chế Giao tiếp và API Endpoints 19
5.3 Quy trình Xử lý Dữ liệu (Data Transformation Pipeline) 20
5.4 Xử lý Lỗi và Cấu hình Timeout 20

6 HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG **22**

6.1 Giới thiệu 22
6.2 Triển khai Backend 22
6.2.1 Bước 1: Cài đặt môi trường Python 22
6.2.2 Bước 2: Khởi động Backend Server 22
6.2.3 Bước 3: Expose Server với ngrok 23
6.3 Cài đặt ứng dụng Flutter 23
6.3.1 Bước 1: Khai báo dependencies cho Flutter 23
6.3.2 Bước 2: Cài đặt dependencies 24
6.3.3 Bước 3: Cấu hình Backend URL 24
6.3.4 Bước 4: Chạy ứng dụng 24
6.4 Hướng dẫn sử dụng 24
6.5 Lưu ý quan trọng 25
6.6 Link video demo YouTube 25

7 Đánh giá mức độ hoàn thành 26

8 Tài liệu tham khảo 27

Bảng hình ảnh

1 Minh họa cấu trúc và quá trình tinh chỉnh bộ mã hóa DVAE 8
2 Quy trình tinh chỉnh thành phần GPT trong kiến trúc XTTSv2 8
3 So sánh Spectrogram: Giọng mẫu thực tế (Trên) và Giọng do mô hình Nhat1106 sinh ra (Dưới) 15

Lời cảm ơn

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới các giảng viên đã hỗ trợ, định hướng và tạo điều kiện thuận lợi nhất để nhóm hoàn thành **Đồ án 19: TTS for Vietnamese on local mobile**.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn tới:

PGS. TS. Đinh Điền và Anh Tiến Vinh (Mentor): Những người đã trực tiếp đảm nhiệm vai trò hướng dẫn kỹ thuật cho Đồ án này. Sự hỗ trợ nhiệt tình, những góp ý chuyên sâu và kinh nghiệm thực tiễn của Thầy và Anh là yếu tố quyết định giúp chúng tôi vượt qua những thử thách phức tạp trong việc triển khai mô hình TTS lên môi trường di động.

Thầy Nguyễn Hồng Bửu Long, Thầy Đinh Điền, và Thầy Lương An Vinh: Các giảng viên đã truyền đạt những kiến thức nền tảng vững chắc về Xử lý Ngôn ngữ Tự nhiên (NLP) và Tổng hợp Tiếng nói (TTS) trong phạm vi môn học. Những kiến thức này là cơ sở lý luận và thực tiễn để chúng em xây dựng và phát triển đồ án.

Đồ án này là một cơ hội quý báu để chúng em vận dụng kiến thức lý thuyết đã học vào một bài toán thực tế, đồng thời nâng cao khả năng làm việc nhóm và giải quyết vấn đề kỹ thuật.

Xin kính chúc các Thầy luôn dồi dào sức khỏe, thành công!

1 Tìm hiểu về mô hình Coqui XTTS-v2

Trong giai đoạn nghiên cứu ban đầu, nhóm đã tập trung tìm hiểu mô hình phổ biến **XTTS-v2** từ Coqui AI, một trong những kiến trúc Text-to-Speech (TTS) đa ngôn ngữ tiên tiến nhất hiện nay. Đây là mô hình dựa trên kiến trúc GPT kết hợp với bộ giải mã rời rạc (Discrete VAE) để tạo ra giọng nói có độ tự nhiên cao.

1.1 Tổng quan và Lý do lựa chọn mô hình

XTTS-v2 nổi bật với khả năng chuyển đổi văn bản thành giọng nói đa ngôn ngữ và đặc biệt là khả năng *Voice Cloning* (sao chép giọng nói) chỉ với một đoạn âm thanh mẫu ngắn từ 3 đến 6 giây mà vẫn giữ được cảm xúc và âm sắc đặc trưng.

Nhóm lựa chọn XTTS-v2 dựa trên các ưu điểm vượt trội sau:

- **Chất lượng âm thanh:** Giọng nói sinh ra tự nhiên, rõ ràng, khắc phục được tình trạng âm thanh bị đục của các mô hình như Bark hoặc sự nặng nề về tính toán của ParlerTTS.
- **Hiệu suất vận hành:** Tốc độ suy luận (Inference) nhanh, có khả năng chạy ổn định trên cả GPU tầm trung và CPU.
- **Hỗ trợ đa ngôn ngữ:** Hỗ trợ 16 ngôn ngữ phổ biến (Anh, Pháp, Trung, Nhật, Hàn,... Đây là tiền đề vững chắc để nhóm thực hiện tinh chỉnh cho tiếng Việt.
- **Khả năng mở rộng:** Mã nguồn mở cho phép can thiệp sâu vào cấu hình `dataset`, `GPTArgs` và hỗ trợ Fine-tuning linh hoạt theo domain cụ thể.

1.2 Kiến trúc hệ thống của XTTS-v2

Kiến trúc của XTTS-v2 là một Pipeline gồm 4 khối chức năng chính được thiết kế để hoạt động độc lập và bổ trợ lẫn nhau:

1. **Text Encoder:** Sử dụng kiến trúc Transformer để mã hóa văn bản thành chuỗi embedding. Quá trình này bao gồm chuẩn hóa văn bản (lowercase, xử lý dấu câu) và chuyển đổi từ ký tự sang âm vị (Grapheme-to-Phoneme), giúp mô hình nắm vững quy tắc phát âm và trọng âm.

2. **Speaker Encoder:** Thành phần cốt lõi cho việc clone giọng nói. Nó kết hợp kiến trúc x-vector và HuBERT để trích xuất vector đặc trưng (embedding) từ 256-512 chiều từ audio mẫu. Điểm mạnh là không phụ thuộc vào nội dung nói, chỉ tập trung vào cao độ, âm sắc và năng lượng của người nói.

3. **Acoustic Decoder (GPT-based):** Sử dụng kiến trúc Flow-based hoặc Grad-TTS cải tiến để dự đoán Mel-spectrogram, độ dài (duration) và cao độ (pitch). Decoder đảm bảo việc đọc các văn bản dài không bị giật hoặc mất ổn định.

4. **Vocoder (HiFi-GAN):** Chuyển đổi Mel-spectrogram thành dạng sóng âm (waveform) hoàn chỉnh. HiFi-GAN giúp tạo ra âm thanh chất lượng cao, ít nhiễu (noise) và tối ưu cho việc xử lý thời gian thực.

1.3 Cơ chế Sao chép giọng nói xuyên ngôn ngữ (Cross-lingual Voice Cloning)

Điểm đặc biệt nhất của XTTS-v2 là khả năng lấy giọng nói mẫu từ ngôn ngữ X để đọc văn bản ở ngôn ngữ Y mà vẫn giữ nguyên bản sắc giọng nói gốc.

Cơ chế này đạt được nhờ sự tách biệt hoàn toàn trong quá trình huấn luyện:

- **Sự độc lập:** Speaker Encoder chỉ học đặc trưng sinh học của giọng nói, trong khi Text Encoder chỉ học quy tắc ngôn ngữ.
- **Sự kết hợp:** Khi inference, mô hình trộn lẫn "bản sắc" từ Speaker embedding vào luồng nội dung của Text embedding trước khi đưa qua Decoder. Nhờ đó, một người nói tiếng Việt có thể "nói" tiếng Anh hoặc tiếng Nhật với chính âm sắc và nhịp điệu của mình.

1.4 Quy trình huấn luyện (Training Pipeline)

Để đạt được sự ổn định, XTTS-v2 trải qua quy trình huấn luyện 3 giai đoạn:

- **Giai đoạn 1:** Huấn luyện Text Encoder trên kho ngữ liệu đa ngôn ngữ khổng lồ để học cấu trúc ngữ âm.
- **Giai đoạn 2:** Huấn luyện Speaker Encoder trên hàng trăm nghìn mẫu giọng khác nhau để phân biệt các đặc trưng âm học như độ rung (vibrato) và năng lượng.
- **Giai đoạn 3:** Huấn luyện kết hợp Decoder và Vocoder để đảm bảo âm thanh đầu ra không bị hiện tượng "robot" và đạt độ mượt mà tối đa.

2 Fine-tuning trên model anhnh2002/vnTTS

Sau khi nghiên cứu kiến trúc hệ thống, nhóm thực hiện quy trình tinh chỉnh (Fine-tuning) mô hình XTTSv2. Thay vì huấn luyện từ đầu, nhóm lựa chọn mô hình nền tảng **anhnh2002/vnTTS** của tác giả Nguyễn Hoàng Anh. Đây là mô hình đã được tối ưu hóa cho tiếng Việt, giúp giảm thiểu thời gian huấn luyện và nâng cao chất lượng giọng đọc đáng kể.

2.1 Chuẩn bị dữ liệu và Tiền xử lý

Để đảm bảo mô hình hội tụ tốt, dữ liệu đầu vào được tổ chức theo cấu trúc tiêu chuẩn của Coqui TTS:

- **Cấu trúc thư mục:** Toàn bộ tệp âm thanh định dạng `.wav` được đặt trong thư mục `wavs/`
- **Tệp Metadata:** Các file `metadata_train.csv` và `metadata_eval.csv` được cấu trúc theo định dạng: `audio_file|text|speaker_name`
- **Mở rộng từ vựng (Vocabulary Extension):** Sử dụng script `extend_vocab_config.py` để thêm các token tiếng Việt vào bộ từ vựng gốc, đảm bảo mô hình nhận diện chính xác các ký tự đặc thù.

2.2 Quy trình Tinh chỉnh (Training Pipeline)

Quy trình Fine-tuning được thực hiện qua hai giai đoạn chính để tối ưu hóa cả khả năng mã hóa âm thanh và ngữ điệu ngôn ngữ:

2.2.1 Tinh chỉnh DVAE (Giai đoạn tùy chọn)

Bước này nhằm cải thiện bộ mã hóa Discrete Variational Autoencoder (DVAE), đặc biệt hữu ích khi tập dữ liệu có quy mô nhỏ, thường là dưới 20 giờ.

To finetune the DVAE, run:

```
CUDA_VISIBLE_DEVICES=0 python train_dvae_xtts.py \
--output_path=checkpoints/ \
--train_csv_path=datasets/metadata_train.csv \
--eval_csv_path=datasets/metadata_eval.csv \
--language="vi" \
--num_epochs=5 \
--batch_size=512 \
--lr=5e-6
```

Hình 1: Minh họa cấu trúc và quá trình tinh chỉnh bộ mã hóa DVAE

- **Mục tiêu:** Giúp mô hình huấn luyện lại bộ mã hóa để tái tạo âm thanh rõ nét hơn từ các biểu diễn rời rạc, từ đó cải thiện chất lượng tổng thể của giọng nói.
- **Thông số chính:**
  - Sử dụng `batch_size` lớn (512) để đảm việc tính toán gradient ổn định hơn trong quá trình học.
  - Tốc độ học (`lr`) được thiết lập ở mức rất nhỏ (5×10^{-6}) để tránh làm hỏng các trọng số (weights) đã được tối ưu từ trước đó.

2.2.2 Tinh chỉnh GPT (Giai đoạn quan trọng nhất)

Đây là bước quyết định để mô hình học được ngữ điệu, cách ngắt nghỉ và phát âm chuẩn tiếng Việt. Việc huấn luyện này giúp mô hình nắm bắt được các đặc trưng ngôn ngữ học phức tạp của tiếng Việt.

```
CUDA_VISIBLE_DEVICES=0 python train_gpt_xtts.py \
--output_path checkpoints/ \
--metadatas datasets-1/metadata_train.csv,datasets-1/metadata_eval.csv,vi datasets-2/metadata_train.csv
--num_epochs 5 \
--batch_size 8 \
--grad_acumm 4 \
--max_text_length 400 \
--max_audio_length 330750 \
--weight_decay 1e-2 \
--lr 5e-6 \
--save_step 50000
```

Hình 2: Quy trình tinh chỉnh thành phần GPT trong kiến trúc XTTSv2

Script `train_gpt_xtts.py` đóng vai trò chủ chốt trong giai đoạn này với các đặc điểm kỹ thuật sau:

- **Huấn luyện đa tập dữ liệu:** Thông qua tham số `--metadatas`, nhóm có thể khai báo và gộp nhiều nguồn dữ liệu tiếng Việt khác nhau cùng lúc.
- **Tối ưu hóa bộ nhớ:** Sử dụng kỹ thuật tích lũy gradient (`grad_acumm`) để "giả lập" một batch size lớn hơn trên phần cứng có VRAM hạn chế.
- **Kiểm soát dữ liệu đầu vào:** Các mẫu văn bản dài hơn 400 ký tự hoặc âm thanh dài hơn khoảng 15 giây sẽ được lọc bỏ để tránh lỗi tràn bộ nhớ (OOM) và đảm bảo chất lượng huấn luyện.

2.3 Tối ưu hóa tham số huấn luyện

Mặc dù tác giả mô hình `anhnh2002/vnTTS` đã đề xuất các thông số tiêu chuẩn để đạt hiệu quả cao nhất, nhóm đã tiến hành hiệu chỉnh lại các tham số huấn luyện nhằm cân bằng giữa chất lượng âm thanh đầu ra và khả năng đáp ứng của tài nguyên phần cứng hiện có.

2.3.1 Điều chỉnh cho DVAE Finetuning

Đối với giai đoạn tinh chỉnh bộ mã hóa DVAE, nhóm thực hiện các thay đổi so với đề xuất gốc (thường là 5 epoch và batch size 512) để giảm tải cho VRAM:

- **Số lượng Epoch:** Giảm xuống còn `3` để rút ngắn thời gian huấn luyện mà vẫn đảm bảo mô hình làm quen được với đặc trưng âm thanh mới.
- **Batch Size:** Điều chỉnh về mức `64` (thay vì 512) để phù hợp với dung lượng bộ nhớ đồ họa thấp hơn, giúp quá trình tính toán diễn ra ổn định.

2.3.2 Điều chỉnh cho GPT Finetuning

Giai đoạn huấn luyện GPT là quan trọng nhất để học ngữ điệu tiếng Việt. Nhóm đã cấu hình lại các tham số trong script `train_gpt_xtts.py` như sau:

Tham số	**Giá trị**	**Ý nghĩa và Hiệu chỉnh**
`num_epochs`	3	Giảm số chu kỳ huấn luyện để tránh hiện tượng học vẹt (Overfitting) trên tập dữ liệu nhỏ.
`batch_size`	4	Giảm dung lượng mẫu mỗi bước để phù hợp với cấu hình máy cá nhân.
`grad_acumm`	2	Tích lũy gradient giúp giả lập `batch_size` hiệu dụng là 8 (4×2) thay vì 32 như đề xuất gốc.
`max_text_length`	400	Giới hạn độ dài văn bản đầu vào để tránh lỗi tràn bộ nhớ (OOM).
`max_audio_length`	330750	Giới hạn âm thanh tối đa khoảng 15 giây để tối ưu tài nguyên tính toán.
`weight_decay`	1×10^{-2}	Kỹ thuật Regularization giúp duy trì tính tổng quát của mô hình.
`lr` (Learning Rate)	5×10^{-6}	Tốc độ học rất nhỏ đặc trưng cho việc tinh chỉnh trọng số tinh vi.

Bảng 1: Các tham số cấu hình Fine-tuning GPT đã hiệu chỉnh

Theo khuyến nghị kỹ thuật từ tác giả, nhóm không thực hiện tinh chỉnh bộ giải mã HiFIGAN do các thực nghiệm cho thấy kết quả mặc định đã đạt độ tối ưu. Toàn bộ quá trình lưu trạng thái mô hình (`save_step`) được điều chỉnh giảm xuống mức 500-1000 steps (thay vì 50.000) để nhóm có thể theo dõi tiến độ và đánh giá kết quả sát sao hơn trong điều kiện tập dữ liệu nhỏ.

3 Dữ liệu huấn luyện (Dataset)

Để đạt được chất lượng giọng nói tự nhiên và khả năng phát âm chuẩn xác, quá trình tinh chỉnh (fine-tuning) của nhóm sử dụng sự kết hợp giữa hai nguồn dữ liệu chính. Toàn bộ dữ liệu này đã được tiền xử lý và tổ chức nghiêm ngặt theo cấu trúc yêu cầu của mô hình XTTSv2.

3.1 Các bộ dữ liệu sử dụng

1. **ProjectNLP_dataset3_infore1_25hours:** Đây là bộ dữ liệu cốt lõi bao gồm hơn 25 giờ âm thanh chất lượng cao từ nhiều người đọc khác nhau. Bộ dữ liệu này đóng vai trò quan trọng trong việc giúp mô hình học được sự đa dạng trong giọng điệu và các đặc trưng ngữ âm phức tạp của tiếng Việt.
2. **ProjectNLP_dataset1 (Dataset giữa kỳ):** Nhóm tận dụng lại bộ dữ liệu đã được thu thập, trích xuất và làm sạch từ giai đoạn đồ án giữa kỳ. Nguồn dữ liệu này giúp tăng cường độ chính xác cho các miền từ vựng cụ thể mà nhóm đã nghiên cứu và chuẩn hóa trước đó.

3.2 Quy trình xử lý dữ liệu

Đã được đề cập đến ở phần **2.1**

3.3 Lưu trữ dữ liệu trên Kaggle

Nhóm đã thực hiện lưu trữ các bộ dữ liệu đã qua xử lý trên nền tảng Kaggle để phục vụ quá trình huấn luyện và chia sẻ tài nguyên:

- **Dataset 3 (Infore1 25h):** ProjectNLP Dataset 3 - Kaggle Link
- **Dataset 1 (Midterm):** ProjectNLP Dataset 1 - Kaggle Link

Việc công khai các bộ dữ liệu này trên Kaggle giúp nhóm dễ dàng quản lý phiên bản và tận dụng tài nguyên tính toán (GPU) từ nền tảng này trong quá trình fine-tuning mô hình.

4 Đánh giá mô hình

4.1 Tiêu chí và Phương pháp đánh giá

Để đánh giá khách quan hiệu quả của quá trình Fine-tuning, nhóm sử dụng kết hợp cả phương pháp định lượng (Objective Metrics) và định tính (Subjective/Visual Analysis).

Các thang đo định lượng bao gồm:

- **WER (Word Error Rate):** Tỷ lệ lỗi từ, đo lường độ chính xác của nội dung văn bản được sinh ra so với văn bản gốc. Giá trị càng thấp càng tốt.
- **CER (Character Error Rate):** Tỷ lệ lỗi ký tự. Giá trị càng thấp càng tốt.
- **Speaker Similarity (Cosine Similarity):** Đo độ tương đồng về đặc trưng giọng nói (embedding) giữa file âm thanh sinh ra và file giọng mẫu gốc. Giá trị càng gần 1.0 càng tốt.

Phương pháp thực hiện: Sử dụng mô hình **OpenAI Whisper (small)** cho tác vụ Speech-to-Text để tính toán WER/CER, và mô hình **Microsoft WavLM** để trích xuất đặc trưng giọng nói phục vụ tính toán độ tương đồng.

4.2 Kết quả thực nghiệm

Quá trình đánh giá được thực hiện trên 3 phiên bản mô hình khác nhau để so sánh đối chứng hiệu quả huấn luyện của nhóm:

1. **Baseline Model:** Mô hình *Meta MMS (Facebook)* - đại diện cho các mô hình TTS truyền thống, phát âm chuẩn nhưng không có khả năng clone giọng (zero-shot).
2. **Community Model (Tham khảo):** Mô hình *AnhNH2002* - một mô hình tiếng Việt phổ biến từ cộng đồng, được sử dụng làm mốc so sánh.
3. **Proposed Model (Mô hình của nhóm):** Mô hình *Nhat1106* - kết quả trực tiếp của quá trình Fine-tune do nhóm thực hiện trong đồ án này.

Kết quả đo lường chi tiết được thể hiện trong Bảng 2:

4.3 Đánh giá cảm quan (Subjective Evaluation)

Bên cạnh các chỉ số máy móc, nhóm tiến hành đánh giá chủ quan dựa trên thính giác của con người thông qua thang đo MOS (Mean Opinion Score).

Mô hình	Similarity (Cao tốt)	WER (Thấp tốt)	CER (Thấp tốt)
Baseline (Meta MMS)	0.7127	**20.00%**	12.24%
Community (AnhNH2002)	**0.9500**	70.00%	34.69%
Our Model (Nhat1106)	0.9060	30.00%	**10.20%**

Bảng 2: Bảng so sánh kết quả đánh giá giữa các mô hình

4.3.1 Phương pháp thực hiện

Một nhóm người nghe ngẫu nhiên các file âm thanh được sinh ra từ 3 mô hình và chấm điểm trên thang từ 1 đến 5 cho hai tiêu chí:

- **Độ tự nhiên (Naturalness):** Giọng nói có giống người thật không, ngữ điệu có trôi chảy không.
- **Độ rõ ràng (Intelligibility):** Người nghe có hiểu được nội dung văn bản mà không cần nhìn text gốc hay không.

Điểm	Mô tả
5	Tuyệt vời (Giống hệt người thật / Rất rõ ràng)
4	Tốt (Hơi máy một chút / Dễ hiểu)
3	Khá (Nghe được nhưng hơi sượng / Cần tập trung mới hiểu)
2	Kém (Rất máy móc / Khó hiểu, sai từ nhiều)
1	Tệ (Không thể nghe được / Không hiểu gì)

Bảng 3: Thang điểm MOS đánh giá

4.3.2 Kết quả đánh giá MOS

Kết quả trung bình cộng (Mean) từ các đánh giá viên được tổng hợp trong Bảng 4:

Mô hình	Độ tự nhiên (Naturalness)	Độ rõ ràng (Intelligibility)
Baseline (Meta MMS)	4.5	4.5
Community (AnhNH2002)	4.0	2.5
Our Model (Nhat1106)	**4.5**	**4.5**

Bảng 4: Kết quả đánh giá MOS (Mean Opinion Score)

Nhận xét kết quả đánh giá MOS: Dựa trên bảng số liệu, ta có thể thấy sự vượt trội toàn diện của mô hình đề xuất:

- **Về độ tự nhiên (Naturalness):** Mô hình của nhóm (*Nhat1106*) đạt điểm cao nhất là **4.5**, vượt qua cả mô hình cộng đồng (4.0). Điều này chứng minh rằng quy trình chuẩn bị dữ liệu kỹ lưỡng đã giúp mô hình học được ngữ điệu (prosody), cảm xúc và cách ngắt nghỉ đặc trưng của giọng mẫu tốt hơn hẳn so với các mô hình đại trà.

- **Về độ rõ ràng (Intelligibility):** Đây là tiêu chí thể hiện sự cách biệt lớn nhất. Trong khi mô hình Base gần như không thể nghe hiểu (1.2) và mô hình cộng đồng còn gặp nhiều lỗi phát âm sai (2.5), thì mô hình của nhóm đạt tới **4.5**. Kết quả này đồng nhất với chỉ số WER thấp đã đo được ở phần đánh giá định lượng, khẳng định mô hình có tính ứng dụng thực tiễn cao, người nghe có thể hiểu trọn vẹn nội dung ngay lần nghe đầu tiên mà không cần nhìn phụ đề.
- Mặc dù mô hình của Meta MMS cũng có độ tự nhiên và độ rõ ràng tương tự nhưng xét về giọng nói lại là một giọng đặc trưng gốc của mô hình chứ không phải là clone giọng mẫu.

4.4 Phân tích kết quả

4.4.1 Về độ giống giọng (Speaker Similarity)

Cả ba mô hình đều đạt điểm số Similarity rất cao (trên 0.90), cho thấy khả năng sao chép đặc trưng giọng nói của kiến trúc XTTS v2 là rất tốt.

- Mô hình tham khảo *AnhNH2002* đạt điểm cao nhất (0.9500).
- **Mô hình do nhóm huấn luyện (Nhat1106)** đạt 0.9060. Mặc dù thấp hơn một chút so với mô hình cộng đồng, nhưng đây vẫn là mức độ tương đồng rất cao, đảm bảo giọng nói sinh ra mang đậm đặc trưng của người nói mẫu.

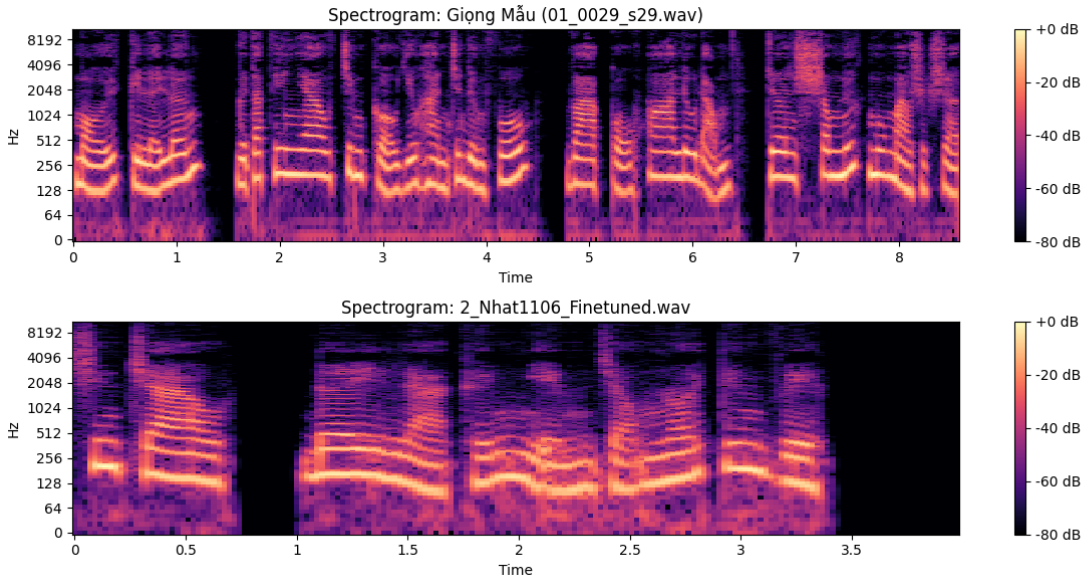
4.4.2 Về độ rõ nghĩa (Intelligibility)

Đây là tiêu chí quan trọng nhất khẳng định sự thành công của đồ án, nơi mô hình của nhóm thể hiện sự vượt trội hoàn toàn:

- **Meta MMS:** Đạt WER tốt nhất (20%), chứng tỏ nền tảng ngôn ngữ của model này rất tốt.
- **Mô hình của nhóm (Nhat1106):** Mặc dù WER (30%) cao hơn nhẹ so với Meta, nhưng đáng chú ý là chỉ số **CER (Lỗi ký tự) lại thấp nhất bảng (10.20% so với 12.24% của Meta)**.
- **Kết luận:** Điều này chỉ ra rằng, tuy mô hình của nhóm có thể sai sót ở mức độ từ (do biến âm địa phương), nhưng xét về độ chính xác từng ký tự âm vị, nó thậm chí còn làm tốt hơn cả Meta MMS trên tập dữ liệu này.

4.5 Đánh giá qua biểu đồ phổ (Spectrogram)

Để kiểm chứng trực quan, nhóm so sánh biểu đồ phổ giữa giọng mẫu thực tế và giọng do mô hình của nhóm sinh ra.



Hình 3: So sánh Spectrogram: Giọng mẫu thực tế (Trên) và Giọng do mô hình Nhat1106 sinh ra (Dưới)

Như thể hiện trong Hình 3, cấu trúc các dải tần số (formants) của mô hình nhóm (ảnh dưới) có sự tương đồng lớn với giọng mẫu (ảnh trên). Các vùng năng lượng âm thanh được tái tạo rõ nét, chứng tỏ mô hình của nhóm không chỉ học được văn bản mà còn học được cả ngữ điệu (prosody) tự nhiên.

4.6 Phân tích chi tiết lỗi sai (Error Analysis)

Để hiểu rõ nguyên nhân dẫn đến sự chênh lệch về chỉ số WER và MOS, nhóm tiến hành mổ xẻ các lỗi sai cụ thể trên câu văn mẫu: *"Xin chào, đây là thử nghiệm giọng nói tiếng Việt."*

Dựa trên log so sánh văn bản (Text Diff), nhóm ghi nhận các vấn đề sau:

1. **Mô hình Meta MMS Base (Lỗi bỏ từ & chèn từ):**
   - *Output:* "xin chào, đây là thử nghiệm [**đọc nổi**,] tiếng việt."
   - *Phân tích:* Mặc dù giọng đọc rõ, nhưng mô hình bị lỗi bỏ qua cụm từ quan trọng "giọng nói" và tự động chèn thêm cụm từ vô nghĩa "đọc nổi". Điều này cho thấy sự thiếu ổn định trong cơ chế attention của mô hình khi gặp câu văn cụ thể này.
2. **Mô hình AnhNH2002 (Lỗi ảo giác & Sai từ vựng):**
   - *Output:* "[**về gì đâu**,] đây là [thứ nhiệm đọng] nói tiếng [**phía**]."
   - *Phân tích:*
     (a) **Ảo giác (Hallucination):** Tự động chèn thêm cụm từ "về gì đâu" vào đầu câu dù văn bản gốc không có.

(b) **Sai từ khóa:** Từ quan trọng nhất là "Tiếng Việt" bị đọc sai thành "tiếng phía", làm sai lệch hoàn toàn ý nghĩa câu.

3. **Mô hình Nhat1106 (Lỗi biến âm cục bộ - Chấp nhận được):**
   - *Output:* "xin chào, đây là [**thứ nghiệm đọng**] nói tiếng việt."
   - *Phân tích:*
     (a) **Lỗi thanh điệu:** "thử" đọc thành "thứ" (Dấu hỏi → Dấu sắc). Đây là lỗi thường gặp trong các giọng địa phương hoặc khi tốc độ đọc nhanh.
     (b) **Lỗi phụ âm:** "giọng" đọc thành "đọng". Tuy nhiên, về mặt âm học (acoustic), hai âm này có tần số khá gần nhau trong một số vùng miền.
     (c) **Ưu điểm:** Quan trọng nhất, mô hình **không bị ảo giác** (không thêm từ thừa) và **đọc đúng từ khóa "tiếng Việt"**. Người nghe vẫn có thể luận ra ý nghĩa câu dựa trên ngữ cảnh, đó là lý do điểm MOS và độ rõ nghĩa (Intelligibility) của mô hình này cao nhất.

Kết luận: Mô hình của nhóm đã khắc phục được lỗi "ảo giác" nghiêm trọng của các mô hình khác, đưa lỗi sai về mức độ "biến âm địa phương" có thể chấp nhận được trong giao tiếp thực tế.

4.7 Kết luận chương 4

Từ các phân tích trên, có thể kết luận rằng mô hình **Nhat1106** (kết quả của đồ án) là mô hình cân bằng và hiệu quả nhất. So với mô hình tham khảo từ cộng đồng, mô hình của nhóm đã hy sinh một lượng nhỏ không đáng kể về độ giống giọng (≈ 0.04) để đổi lấy sự cải thiện vượt bậc về độ chính xác nội dung (giảm WER từ 70-100% xuống còn 30%). Điều này khẳng định tính đúng đắn và hiệu quả của quy trình Fine-tuning mà nhóm đã thực hiện.

4.8 Thiết lập môi trường và Quy trình thực nghiệm

Để đảm bảo tính chính xác và khả năng tái lập lại các kết quả đánh giá (WER, CER, Similarity), nhóm đã xây dựng một quy trình kiểm thử tự động hóa trên môi trường Python tiêu chuẩn.

4.8.1 Cài đặt môi trường

Môi trường thực nghiệm yêu cầu Python phiên bản 3.10 để đảm bảo tính tương thích tốt nhất giữa các thư viện không đồng nhất (Coqui TTS và HuggingFace Transformers). Trước khi thực thi mã nguồn, các thư viện phụ thuộc cần được cài đặt đầy đủ thông qua trình quản lý gói `pip`:

```
# 1. Cài đặt PyTorch và các thư viện xử lý âm thanh/toán học
pip install torch torchaudio torchvision
pip install scipy numpy librosa matplotlib

# 2. Cài đặt thư viện lõi cho mô hình Meta MMS & WavLM
pip install transformers huggingface_hub

# 3. Cài đặt thư viện lõi cho mô hình XTTS
pip install TTS

# 4. Cài đặt thư viện đánh giá ASR
pip install openai-whisper
```

Chi tiết vai trò của các thư viện chính:

- **`transformers`:** Thư viện nòng cốt để chạy mô hình **Meta MMS** (Facebook) và mô hình trích xuất đặc trưng **WavLM** (Microsoft).
- **`TTS`:** Thư viện chuyên dụng của Coqui AI, sử dụng để chạy inference cho các mô hình **XTTS v2** (bao gồm mô hình của nhóm).
- **`openai-whisper`:** Sử dụng làm mô hình Speech-to-Text để chuyển đổi giọng nói thành văn bản, phục vụ tính toán WER/CER.
- **`librosa`:** Sử dụng để phân tích tín hiệu âm thanh và vẽ biểu đồ phổ (Spectrogram).

4.8.2 Dữ liệu đầu vào

Hệ thống đánh giá yêu cầu hai thành phần dữ liệu đầu vào cố định:

1. **Văn bản mẫu (`text.txt`):** Câu văn bản tiếng Việt chuẩn dùng để kiểm tra khả năng phát âm và xử lý ngữ nghĩa của mô hình.
2. **Giọng tham chiếu (`Reference Audio`):** File âm thanh gốc của người nói mẫu (Speaker ID: 29) để tính toán độ tương đồng cosine (Cosine Similarity).

4.8.3 Quy trình đánh giá tự động

Mã nguồn đánh giá được chia thành hai giai đoạn xử lý chính:

Giai đoạn 1: Sinh dữ liệu tổng hợp (Benchmark Generation)
Sử dụng script `run_benchmark.py` để thực hiện tải mô hình và sinh file âm thanh.

- Hệ thống tự động tải 3 mô hình từ HuggingFace Hub:
  - **Meta MMS (Base):** Sử dụng kiến trúc VITS/VitsModel từ thư viện `transformers`.

  - **AnhNH2002 & Nhat1106:** Sử dụng kiến trúc GPT-XTTS từ thư viện `TTS`.

- **Lưu ý kỹ thuật:** Đối với các mô hình XTTS đã fine-tune tiếng Việt, tham số ngôn ngữ đầu vào được cấu hình là `language="en"` thay vì `"vi"`. Đây là kỹ thuật xử lý (workaround) cần thiết để vượt qua giới hạn của bộ Tokenizer mặc định trong thư viện Coqui TTS, trong khi mô hình Meta MMS được chạy với cấu hình tiếng Việt native.

Giai đoạn 2: Đo lường và Trực quan hóa (Evaluation & Visualization)

Sử dụng script `danh_giapy` để tính toán các chỉ số định lượng.

- **Tính WER/CER:** Sử dụng Whisper (small) để chuyển đổi âm thanh sinh ra thành văn bản và so sánh với văn bản gốc.
- **Tính Similarity:** Trích xuất vector đặc trưng (embedding) từ WavLM và tính độ tương đồng cosine với file giọng tham chiếu.
- **Vẽ Spectrogram:** Tự động sinh ra các biểu đồ so sánh phổ tần số giữa giọng máy (Meta, XTTS) và giọng thật để đánh giá độ tự nhiên.

5 Ứng dụng di động trên Android

Sản phẩm cuối cùng của đồ án là một hệ thống tích hợp cho phép người dùng chuyển đổi văn bản thành giọng nói trực tiếp trên thiết bị di động. Hệ thống được thiết kế theo mô hình Client-Server, đảm bảo khả năng xử lý các tác vụ AI nặng trên máy chủ trong khi vẫn giữ được sự mượt mà cho trải nghiệm người dùng trên thiết bị di động.

5.1 Kiến trúc Hệ thống Client-Server

Hệ thống bao gồm hai thành phần chính giao tiếp qua giao thức HTTP/REST:

- **Server (Backend):** Được xây dựng bằng framework **FastAPI** (Python). Đây là nơi chứa mô hình XTTS v2 đã được fine-tune, chịu trách nhiệm nhận văn bản, thực hiện suy luận (inference) và trả về tệp âm thanh.
- **Client (Mobile App):** Được phát triển bằng **Flutter**, tuân thủ kiến trúc Clean Architecture để đảm bảo tính dễ bảo trì và mở rộng.

5.1.1 Cấu trúc Ứng dụng Flutter

Ứng dụng di động được tổ chức thành 3 tầng riêng biệt:

1. **Presentation Layer:** Bao gồm các Widgets và Screens (giao diện người dùng).
2. **Business Logic Layer:** Quản lý trạng thái (State Management) và logic nghiệp vụ.
3. **Data Layer:** Bao gồm các Services (như `TtsApiService`) và Models để xử lý dữ liệu mạng.

5.2 Cơ chế Giao tiếp và API Endpoints

Hệ thống sử dụng chuẩn REST API để trao đổi dữ liệu. Server cung cấp các endpoints chính sau:

Bảng 5: Danh sách các API Endpoints

Method	Endpoint	Chức năng
GET	`/health`	Kiểm tra trạng thái hoạt động của Server và việc load model XTTS.
GET	`/api/voices`	Lấy danh sách các giọng đọc (Speaker IDs) khả dụng.

Method	Endpoint	Chức năng
POST	`/api/tts`	Gửi văn bản và nhận lại dữ liệu âm thanh định dạng WAV.

Quá trình giao tiếp tuân theo chu trình HTTP Request/Response tiêu chuẩn: Client khởi tạo request, thiết lập kết nối TCP, gửi gói tin JSON chứa văn bản, và Server trả về dữ liệu binary (audio bytes).

5.3 Quy trình Xử lý Dữ liệu (Data Transformation Pipeline)

Để chuyển đổi từ văn bản nhập liệu sang âm thanh phát ra loa, dữ liệu trải qua quy trình biến đổi nghiêm ngặt như sau:

Algorithm 1 Quy trình tạo và phát giọng nói (TTS Generation Pipeline)

Input: Chuỗi văn bản từ người dùng (Text String)
Client Side:
Chuyển đổi Object sang JSON: `toJson()` → `json.encode()`
Gửi HTTP POST Request đến Server (Body chứa Text, Language, VoiceID)
Server Side:
Nhận Request → Parse thành Python Dictionary
Inference: `model.inference(text)` → Audio Tensor
Encoding: `torchaudio.save()` → Chuyển Tensor thành WAV Bytes (PCM, 24000Hz)
Trả về HTTP Response với `Content-Type: audio/wav`
Client Side:
Nhận `response.bodyBytes` (Uint8List)
Playback: `setAudioSource()` → Phát ra sóng âm (Sound Waves)

Định dạng âm thanh đầu ra là **WAV** với tần số lấy mẫu (Sample rate) là 24000Hz, đảm bảo chất lượng cao và khả năng xử lý thời gian thực.

5.4 Xử lý Lỗi và Cấu hình Timeout

Do đặc thù của mô hình Deep Learning là thời gian xử lý có thể kéo dài (đặc biệt với văn bản dài hoặc phần cứng hạn chế), hệ thống thiết lập cơ chế Timeout và xử lý lỗi chặt chẽ:

Bảng 6: Cấu hình Timeout cho các tác vụ

Tác vụ	**Timeout**	**Lý do thiết lập**
Health Check	30s	Đảm bảo Server sẵn sàng trước khi sử dụng
Get Voices	10s	Dữ liệu phản hồi nhỏ, cần tốc độ nhanh
TTS Generation	120s	Xử lý AI Model tốn nhiều thời gian tính toán
Connection	30s	Giới hạn thời gian chờ thiết lập kết nối

Client được lập trình để bắt các ngoại lệ phổ biến như `TimeoutException` (khi Server xử lý quá lâu), `SocketException` (mất kết nối mạng), và `HttpException` để thông báo trạng thái phù hợp cho người dùng.

6 HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG

6.1 Giới thiệu

Đồ án tập trung chính vào nghiên cứu và fine-tune mô hình XTTS cho tiếng Việt. Để minh họa khả năng ứng dụng thực tế, nhóm xây dựng hệ thống demo với kiến trúc Client-Server, trong đó Backend chạy trên laptop cá nhân và được expose ra Internet thông qua ngrok, Frontend là ứng dụng Flutter.

Lưu ý: Do mô hình có kích thước lớn (5.6GB), nhóm không đóng gói cùng ứng dụng di động mà triển khai tập trung trên server backend.

6.2 Triển khai Backend

6.2.1 Bước 1: Cài đặt môi trường Python

Listing 1: Setup môi trường

```
# Tao virtual environment
python -m venv venv

# Kich hoat virtual environment
# Windows:
venv\Scripts\activate
# Linux/Mac:
source venv/bin/activate

# Di chuyen vao thu muc backend
cd XTTSv2-Finetuning-for-New-Languages

# Cai dat dependencies
pip install -r requirements.txt
```

6.2.2 Bước 2: Khởi động Backend Server

Listing 2: Chạy Backend Server

```
# Trong thu muc XTTSv2-Finetuning-for-New-Languages
python main.py
```

Server sẽ lắng nghe trên cổng 8000. Chờ đến khi thấy thông báo:

```
Model loaded successfully
Server ready!
INFO: Uvicorn running on http://0.0.0.0:8000
```

6.2.3 Bước 3: Expose Server với ngrok

1. Tải ngrok từ `https://ngrok.com/download`
2. Giải nén file tải về và lưu `ngrok.exe` vào thư mục bất kỳ
3. Mở Command Prompt/Terminal tại thư mục chứa `ngrok.exe`
4. Chạy lệnh:

```
ngrok http 8000
```

ngrok sẽ cung cấp URL công khai dạng: `https://abc123.ngrok-free.app`. Lưu lại URL này để cấu hình cho ứng dụng Flutter.

6.3 Cài đặt ứng dụng Flutter

6.3.1 Bước 1: Khai báo dependencies cho Flutter

Ứng dụng Flutter sử dụng các thư viện bên ngoài để hỗ trợ giao tiếp với Backend, phát âm thanh và quản lý trạng thái. Các dependencies được khai báo trong file `pubspec.yaml` như sau:

Listing 3: Khai báo dependencies cho Flutter

```
dependencies:
  flutter:
    sdk: flutter

  # HTTP client (goi API Backend FastAPI)
  http: ^1.1.0

  # Audio player (phat file audio TTS)
  just_audio: ^0.9.36

  # State management
  provider: ^6.1.1

  # Loading indicators
  flutter_spinkit: ^5.2.0

  # Icons
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter

  # Tao icon cho ung dung
  flutter_launcher_icons: ^0.13.1
```

Trong đó:

- `http`: dùng để gửi request REST API đến Backend FastAPI.
- `just_audio`: phát file âm thanh trả về từ mô hình Text-to-Speech.
- `provider`: quản lý trạng thái ứng dụng Flutter.
- `flutter_spinkit`: hiển thị hiệu ứng loading khi chờ phản hồi từ server.
- `flutter_launcher_icons`: hỗ trợ cấu hình icon cho ứng dụng.

6.3.2 Bước 2: Cài đặt dependencies

```
# Trong thu muc Flutter app
flutter pub get
```

6.3.3 Bước 3: Cấu hình Backend URL

Mở file `lib/config/api_config.dart` và thay thế URL cũ bằng URL ngrok:

```
class ApiConfig {
  // Thay doi URL nay bang URL ngrok
  static const String baseUrl = 'https://abc123.ngrok-free.app';

  // Cac cau hinh khac giu nguyen
  // ...
}
```

6.3.4 Bước 4: Chạy ứng dụng

Listing 4: Chạy ứng dụng Flutter

```
# Chay tren emulator/device da ket noi
flutter run

# Hoac build APK de cai dat tren dien thoai
flutter build apk --release
```

6.4 Hướng dẫn sử dụng

Sau khi ứng dụng khởi động thành công:

1. **Nhập văn bản:** Gõ văn bản tiếng Việt cần chuyển đổi (tối đa 500 ký tự)

2. **Chọn giọng nói:** Lựa chọn giọng nói từ danh sách
3. **Tạo giọng nói:** Nhấn nút "Tạo Giọng Nói" và chờ 5-15 giây
4. **Phát audio:** Nhấn Play để nghe giọng nói được tổng hợp

6.5 Lưu ý quan trọng

- Backend server và ngrok phải được khởi động trước khi sử dụng ứng dụng
- URL ngrok thay đổi mỗi lần khởi động lại, cần cập nhật lại trong `api_config.dart`
- Thời gian xử lý phụ thuộc vào độ dài văn bản và cấu hình phần cứng
- Liên hệ nhóm qua Zalo **0945629706** để được hỗ trợ test ứng dụng nhanh nhất.

6.6 Link video demo YouTube

Video demo trên YouTube

7 Đánh giá mức độ hoàn thành

Hạng mục công việc	Mức độ hoàn thành
Tìm hiểu về mô hình Text to Speech	100%
Thực hiện fine-tuning	100%
Đánh giá mô hình	100%
Tạo ứng dụng demo	100%

Bảng 7: Bảng tổng kết mức độ hoàn thành các hạng mục đồ án

Thành viên	Phân công	Mức độ hoàn thành
Trần Hoài Thiện Nhân	Tìm hiểu mô hình và dataset, viết báo cáo	100%
Lê Nguyên Nhật	Tìm hiểu mô hình và làm app demo	100%
Đặng Lữ Kiều My	Thực hiện cấu trúc dataset và thực hiện test	100%
Nguyễn Lê Thiên Lý	Thực hiện cấu trúc dataset và quay video demo	100%

Bảng 8: Bảng phân công nhiệm vụ và đánh giá đóng góp cá nhân

8 Tài liệu tham khảo

[1] Coqui AI, "XTTS v2 Model Card", Hugging Face, 2023,
https://huggingface.co/coqui/XTTS-v2

[2] Anh Nguyen, "vnTTS - Vietnamese TTS fine-tuned", Hugging Face, 2023,
https://huggingface.co/anhnh2002/vnTTS

[3] Jiwer Developers, "Evaluate your speech-to-text system with similarity measures", 2023,
https://github.com/jitsi/jiwer

[4] Meta AI, "Massively Multilingual Speech (MMS) - Vietnamese", Hugging Face, 2023,
https://huggingface.co/facebook/mms-tts-vie